

Nome: Rafael Gabriel Sehn do Nascimento

Projeto: Atividade N3 Banco de Dados

Professor: Willian Hübner

Link do GitHub: <https://github.com/Rafael-Sehn/gerenciador-de-jogos>

Documentação do Banco de Dados sobre GERENCIADOR DE JOGOS

1. Introdução

Este trabalho apresenta o desenvolvimento de um banco de dados em SQLite para um sistema chamado Gerenciador de Jogos. A ideia do projeto foi criar uma estrutura simples, mas funcional, parecida com um pequeno ERP para uma loja de jogos. O banco foi montado no DB Browser for SQLite e possui tabelas para clientes, jogos, categorias, plataformas, fornecedores, vendas, itens da venda e pagamentos. Também foram criadas regras de integridade, relacionamentos entre tabelas, triggers e relatórios em SQL. Para os testes, foram cadastrados jogos conhecidos, como Minecraft, Grand Theft Auto V, Elden Ring, God of War Ragnarök, The Legend of Zelda: Breath of the Wild, Mario Kart 8 Deluxe, Street Fighter 6, Resident Evil 4 Remake, Red Dead Redemption 2, The Witcher 3: Wild Hunt, Hollow Knight e Stardew Valley.

2. Objetivo do projeto

O objetivo principal foi projetar e testar um banco de dados capaz de controlar informações básicas de uma loja de jogos. Com esse banco é possível cadastrar produtos, organizar plataformas e categorias, registrar vendas, controlar pagamentos e acompanhar o estoque. Além disso, o projeto também teve como objetivo aplicar os recursos estudados em sala, como chaves primárias, chaves estrangeiras, restrições de campos, comandos SQL, joins, agrupamentos, funções de agregação e triggers.

3. Tema escolhido

O tema escolhido foi um gerenciador para loja de jogos. Esse tema foi selecionado porque permite trabalhar com várias partes comuns de um ERP, como clientes, produtos, fornecedores, vendas, pagamentos e estoque. Dessa forma, o banco não fica limitado a um cadastro simples. Ele permite simular uma rotina básica de venda: o cliente compra um ou mais jogos, os itens são lançados na venda, o estoque é atualizado e o pagamento é registrado.

4. Organização do banco de dados

Foram utilizados padrões de nome para deixar o banco mais organizado e fácil de entender. As tabelas foram criadas no plural, os campos seguem o padrão snake_case e as chaves primárias usam o formato id_nome.

Padrão utilizado	Exemplo no banco
Tabelas no plural	clientes, jogos, vendas, pagamentos, categorias, plataformas, fornecedores
Chave primária com id_	id_cliente, id_jogo, id_venda, id_pagamento
Chave estrangeira mantendo o mesmo nome	id_cliente em vendas, id_jogo em venda_itens
Campos em snake_case	preco_venda, estoque_atual, estoque_minimo, forma_pagamento
Campos de controle	status e ativo

5. Diagrama de relacionamentos e funcionamento

O diagrama a seguir mostra como as tabelas se relacionam. Um cliente pode ter várias vendas. Uma venda pode ter vários itens e pagamentos. Cada item da venda está ligado a um jogo. Cada jogo possui categoria, plataforma e fornecedor.

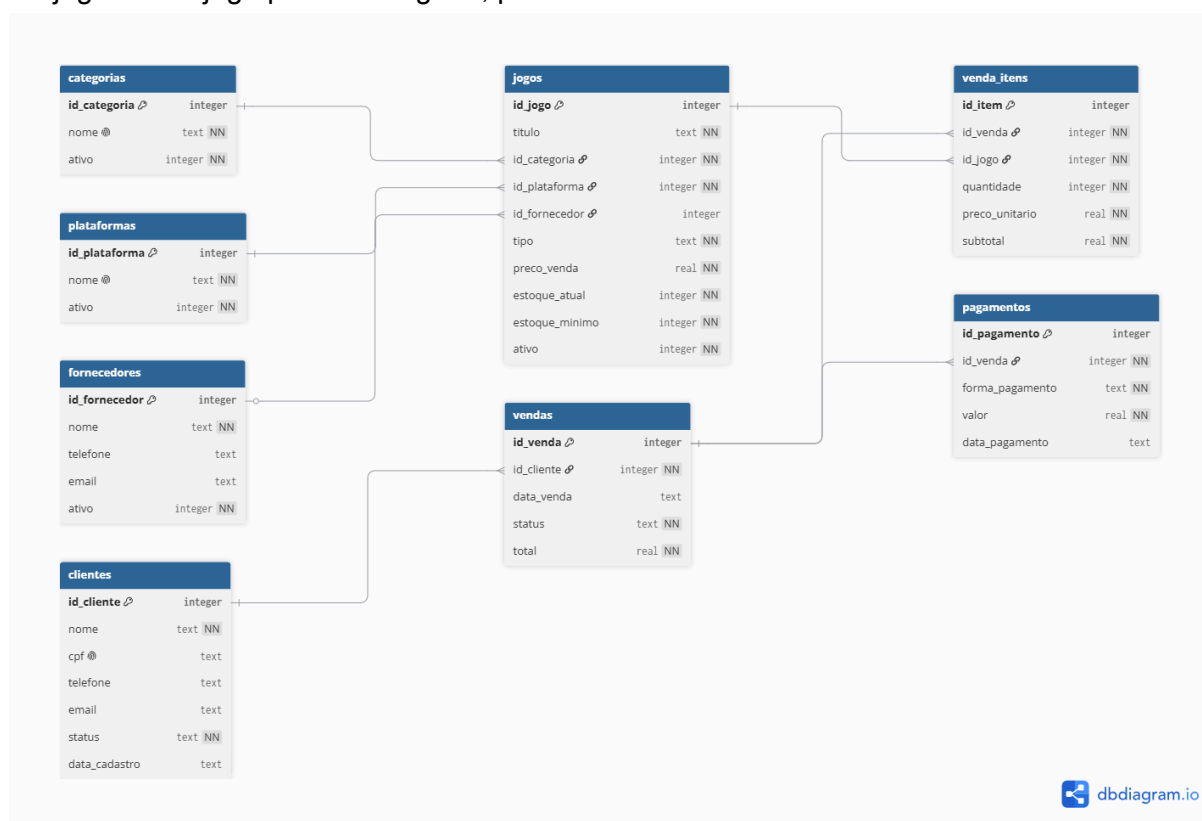


Figura 1 - Diagrama de relacionamentos do banco de dados.

6. Explicação das tabelas

Tabela	Descrição
clientes	Guarda os dados dos clientes, como nome, CPF, telefone, e-mail, status e data de cadastro.
categorias	Armazena os gêneros dos jogos, por exemplo RPG, ação, corrida, luta, estratégia, esporte, aventura e terror.
plataformas	Registra as plataformas em que os jogos podem ser vendidos, como PC, PlayStation 5, Xbox Series X/S, Nintendo Switch e Mobile.
fornecedores	Guarda os dados dos fornecedores responsáveis pelos jogos cadastrados.
jogos	Tabela de produtos. Contém título, categoria, plataforma, fornecedor, tipo do jogo, preço de venda, estoque atual e estoque mínimo.
vendas	Registra cada venda feita para um cliente, com data, status e total.
venda_itens	Registra quais jogos fazem parte de cada venda, com quantidade, preço unitário e subtotal.
pagamentos	Registra a forma de pagamento e o valor pago em cada venda.
sqlite_sequence	Tabela interna criada automaticamente pelo SQLite para controlar campos AUTOINCREMENT.

8. Triggers utilizadas

Foram criadas duas triggers para automatizar regras importantes no momento da venda. Elas são necessárias para o sistema não depender apenas de alteração manual.

8.1 Trigger para verificar estoque

A trigger `trg_verifica_estoque` roda antes de inserir um item na venda. Ela confere se o jogo possui estoque suficiente. Se não houver estoque, a venda do item é bloqueada.

8.2 Trigger para atualizar venda e estoque

A trigger `trg_atualiza_venda_estoque` roda depois de inserir um item na venda. Ela calcula o subtotal, reduz o estoque do jogo e atualiza o total da venda.

9. Relatórios SQL

Foram criados cinco relatórios para consultar informações úteis do sistema. Esses relatórios também comprovam a utilização dos comandos obrigatórios pedidos na atividade.

Relatório 1 - Vendas por cliente

Este relatório mostra o nome do cliente, a quantidade de vendas finalizadas e o valor total gasto. Ele ajuda a ver quais clientes compraram mais. Recursos utilizados: JOIN, WHERE, AND, COUNT e SUM.

The screenshot shows the DB Browser for SQLite interface. The SQL editor contains the following query:

```
1 SELECT
2   c.nome AS cliente,
3   COUNT(v.id_venda) AS quantidade_vendas,
4   SUM(v.total) AS total_gasto
5 FROM vendas v
6 JOIN clientes c ON c.id_cliente = v.id_cliente
7 WHERE v.status = 'finalizada'
8 AND c.status = 'ativo'
9 GROUP BY c.id_cliente, c.nome
```

The results are displayed in a table with 5 rows:

	cliente	quantidade_vendas	total_gasto
1	Rafael Santos	1	579.8
2	Carlos Henrique	1	479.8
3	Mariana Souza	1	449.8
4	Lucas Almeida	1	379.8
5	Ana Paula	1	109.7

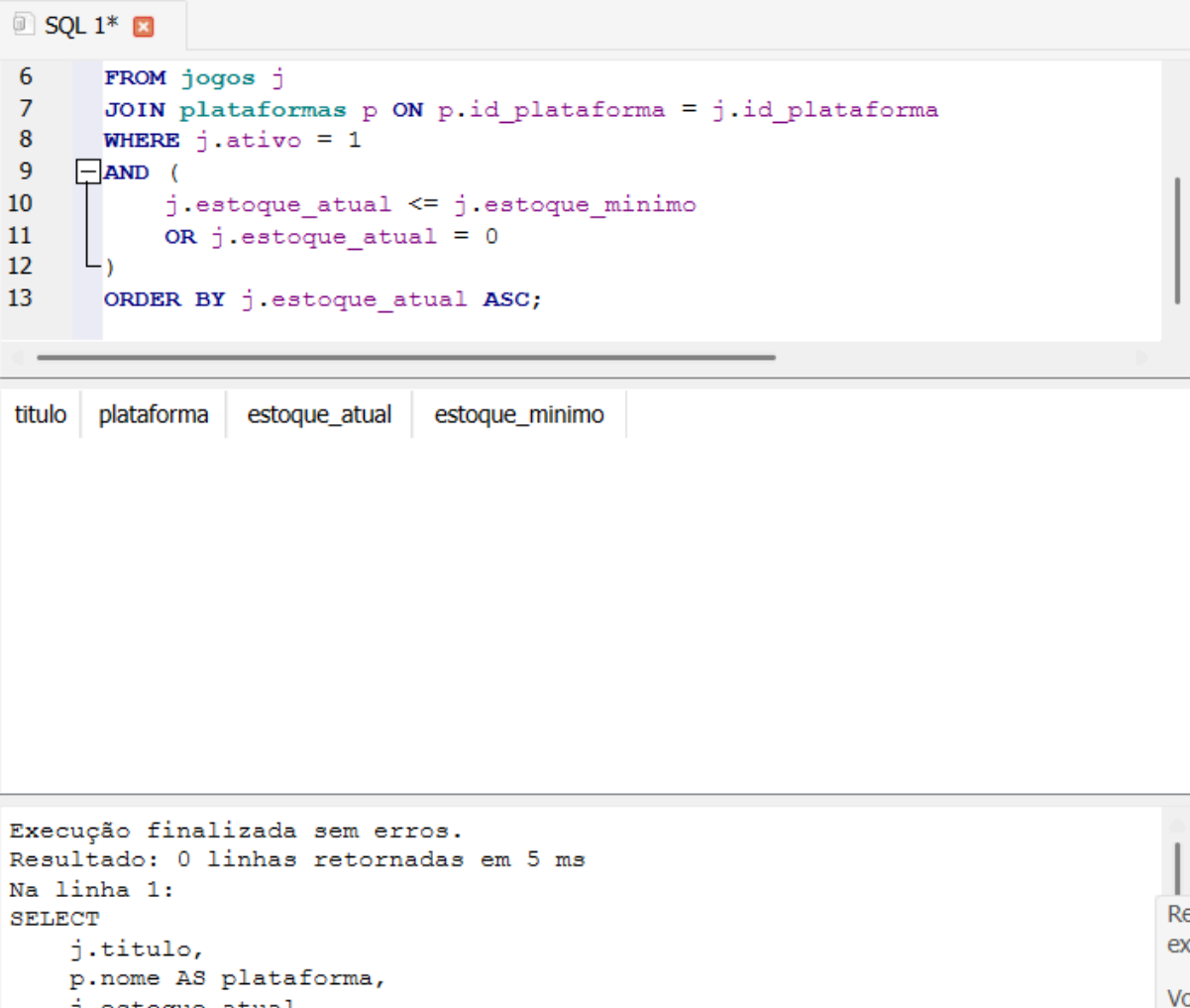
Below the table, the execution status is shown: "Execução finalizada sem erros. Resultado: 5 linhas retornadas em 7 ms. Na linha 1: SELECT c.nome AS cliente, COUNT(v.id_venda) AS quantidade_vendas, SUM(v.total) AS total_gasto".

The right sidebar shows the "Editar célula do banco de dados" panel with a "Modo: Texto" dropdown and a "NULL" value. Below it, the "Remoto" section shows a connection to "DBHub.io" with "Local" and "Current Database" tabs. The bottom of the sidebar has buttons for "Log do SQL", "Plotar", "Esquema do banco de dados", and "Remoto".

Figura 2 - Resultado do relatório de vendas por cliente.

Relatório 2 - Jogos com estoque baixo

Este relatório lista jogos ativos com estoque atual menor ou igual ao estoque mínimo, ou com estoque zerado. No teste, a consulta retornou 0 linhas, o que mostra que nenhum jogo estava abaixo do mínimo no momento. Recursos utilizados: JOIN, WHERE, AND e OR.



The screenshot shows a SQL IDE window titled "SQL 1*" with a query editor and a results pane. The query editor contains the following SQL code:

```
6 FROM jogos j
7 JOIN plataformas p ON p.id_plataforma = j.id_plataforma
8 WHERE j.ativo = 1
9 AND (
10     j.estoque_atual <= j.estoque_minimo
11     OR j.estoque_atual = 0
12 )
13 ORDER BY j.estoque_atual ASC;
```

The results pane shows the execution status and the result set. The execution status is "Execução finalizada sem erros." and the result is "Resultado: 0 linhas retornadas em 5 ms". The result set is empty, showing only the column headers:

titulo	plataforma	estoque_atual	estoque_minimo
--------	------------	---------------	----------------

On the right side of the results pane, there is a vertical scrollbar and a small menu with the options "Re", "ex", and "Vo".

Figura 3 - Resultado do relatório de estoque baixo, sem erros e com retorno de 0 linhas.

Relatório 3 - Faturamento por plataforma

Este relatório agrupa as vendas por plataforma e mostra a quantidade de itens vendidos e o faturamento total. Assim, é possível saber qual plataforma vendeu mais no teste.

Recursos utilizados: JOIN, WHERE, COUNT e SUM.

SQL 1*

```
4      SUM(vi.subtotal) AS faturamento_total
5  FROM venda_itens vi
6  JOIN jogos j ON j.id_jogo = vi.id_jogo
7  JOIN plataformas p ON p.id_plataforma = j.id_plataforma
8  JOIN vendas v ON v.id_venda = vi.id_venda
9  WHERE v.status = 'finalizada'
10 GROUP BY p.id_plataforma, p.nome
11 ORDER BY faturamento_total DESC;
```

	plataforma	quantidade_itens_vendidos	faturamento_total
1	PlayStation 5	5	1159.5
2	Nintendo Switch	2	599.8
3	PC	3	239.6

Execução finalizada sem erros.

Resultado: 3 linhas retornadas em 7 ms

Na linha 1:

```
SELECT
    p.nome AS plataforma,
    COUNT(vi.id_item) AS quantidade_itens_vendidos,
    SUM(vi.subtotal) AS faturamento_total
```

Figura 4 - Resultado do relatório de faturamento por plataforma.

Relatório 4 - Jogos físicos e digitais

Este relatório separa os jogos físicos e digitais em uma única consulta. Foi usado UNION ALL para juntar os dois resultados sem remover registros. Recursos utilizados: UNION ALL e WHERE.

```
10
11 SELECT
12     'Jogos Digitais' AS tipo_relatorio,
13     titulo,
14     preco_venda,
15     estoque_atual
16 FROM jogos
17 WHERE tipo = 'digital';
```

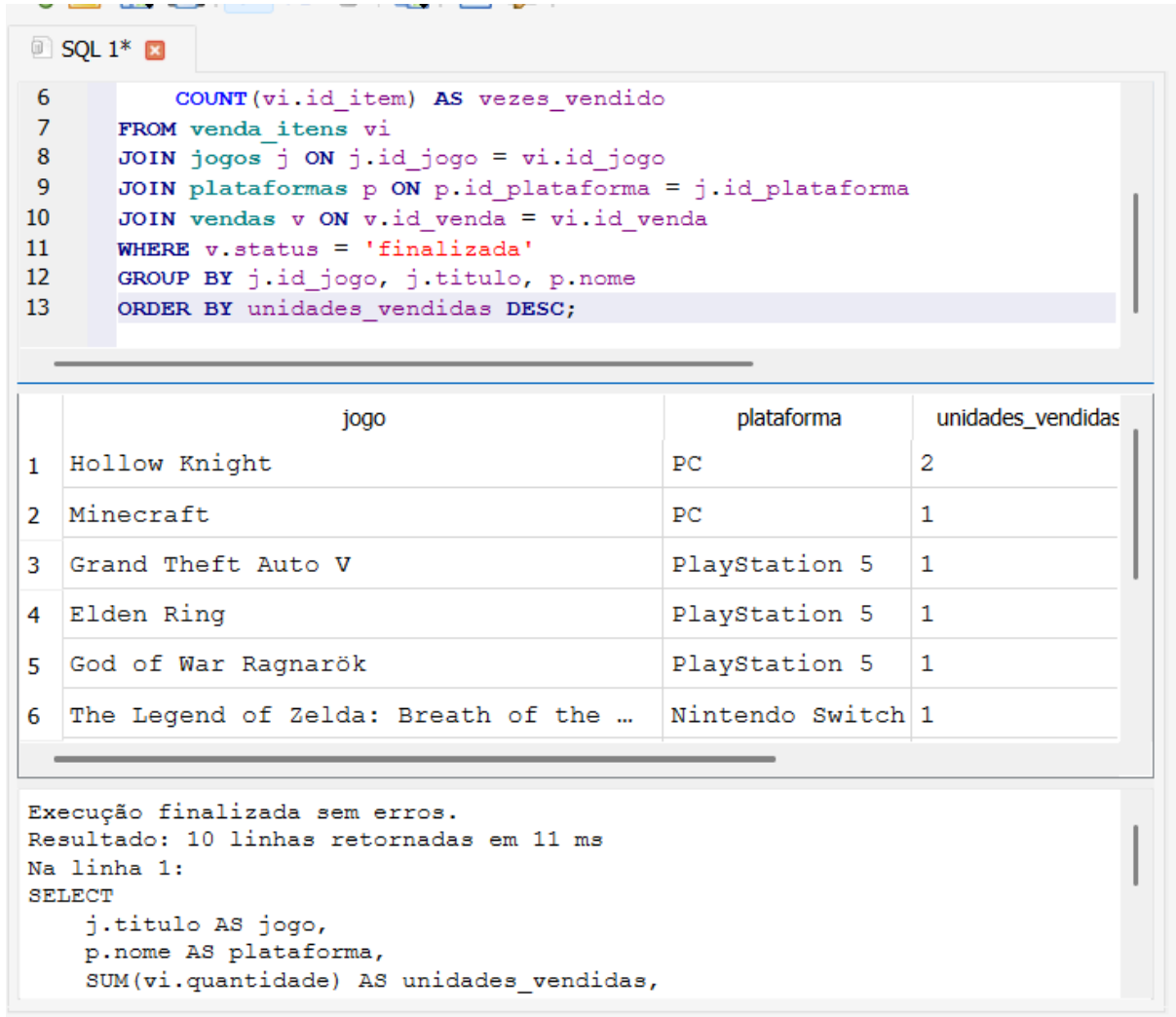
	tipo_relatorio	titulo	preco_venda	esto
1	Jogos Físicos	Grand Theft Auto V	149.9	9
2	Jogos Físicos	Elden Ring	249.9	7
3	Jogos Físicos	God of War Ragnarök	279.9	5
4	Jogos Físicos	The Legend of Zelda: Breath of the ...	299.9	4
5	Jogos Físicos	Mario Kart 8 Deluxe	299.9	6
6	Jogos Físicos	Street Fighter 6	249.9	8

Execução finalizada sem erros.
Resultado: 12 linhas retornadas em 10 ms
Na linha 1:
SELECT
 'Jogos Físicos' AS tipo_relatorio,
 titulo,
 preco_venda,

Figura 5 - Resultado do relatório de jogos físicos e digitais.

Relatório 5 - Jogos mais vendidos

Este relatório mostra os jogos mais vendidos por quantidade de unidades. Também mostra a plataforma, o valor total vendido e quantas vezes o jogo apareceu em itens de venda. Gerenciador de Jogos - Banco de Dados Página 12 Recursos utilizados: JOIN, WHERE, COUNT e SUM.



The screenshot shows a SQL query editor with a query that counts the number of times each game appears in sales items, grouped by game title and platform, and ordered by the number of units sold in descending order. The results are displayed in a table with 6 rows. Below the table, the execution status is shown as 'Execução finalizada sem erros.' followed by the number of lines returned and the execution time. A preview of the SQL query is also shown at the bottom.

```
6      COUNT(vi.id_item) AS vezes_vendido
7  FROM venda_itens vi
8  JOIN jogos j ON j.id_jogo = vi.id_jogo
9  JOIN plataformas p ON p.id_plataforma = j.id_plataforma
10 JOIN vendas v ON v.id_venda = vi.id_venda
11 WHERE v.status = 'finalizada'
12 GROUP BY j.id_jogo, j.titulo, p.nome
13 ORDER BY unidades_vendidas DESC;
```

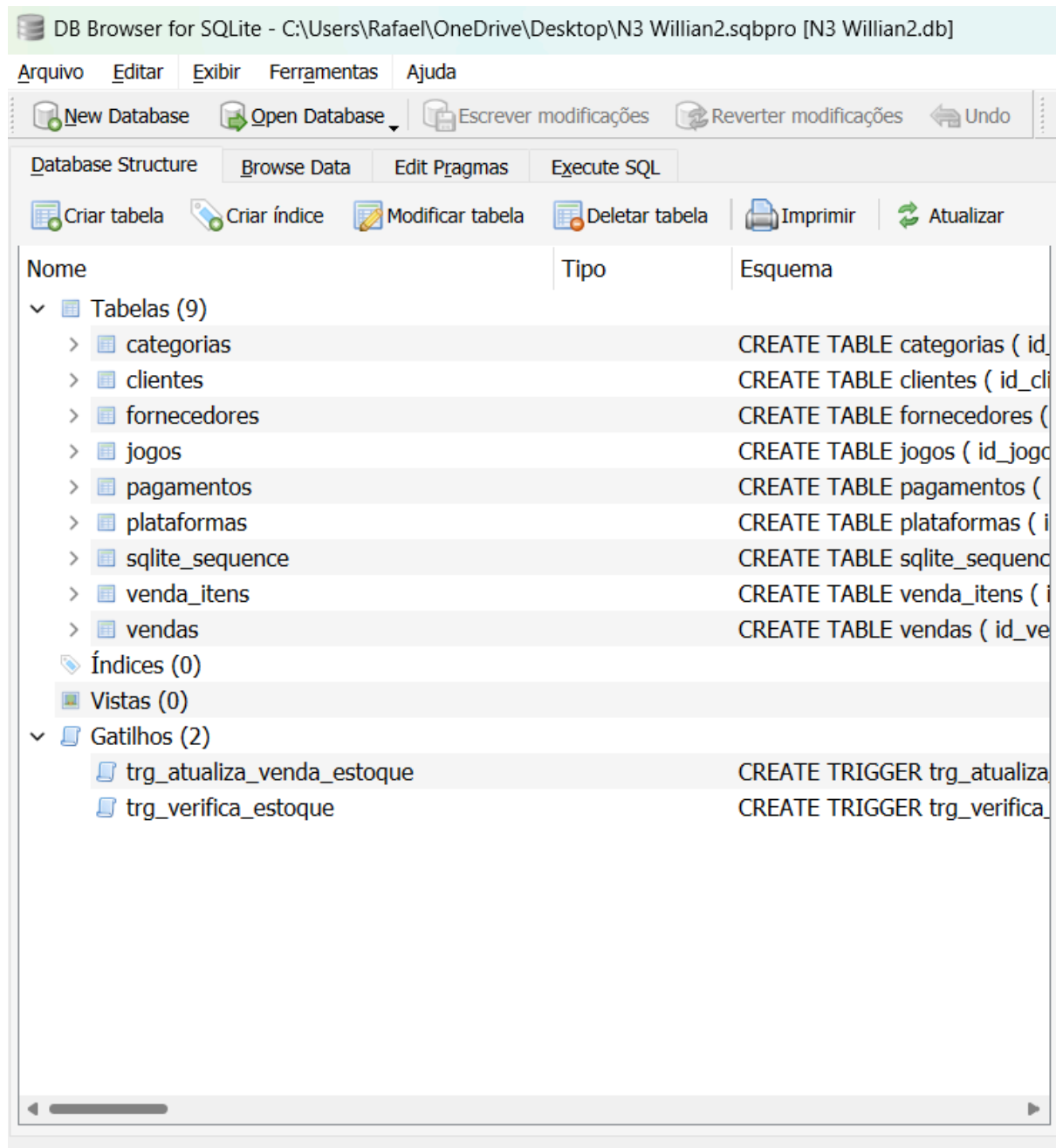
	jogo	plataforma	unidades_vendidas
1	Hollow Knight	PC	2
2	Minecraft	PC	1
3	Grand Theft Auto V	PlayStation 5	1
4	Elden Ring	PlayStation 5	1
5	God of War Ragnarök	PlayStation 5	1
6	The Legend of Zelda: Breath of the ...	Nintendo Switch	1

Execução finalizada sem erros.
Resultado: 10 linhas retornadas em 11 ms
Na linha 1:
SELECT
 j.titulo AS jogo,
 p.nome AS plataforma,
 SUM(vi.quantidade) AS unidades_vendidas,

Figura 6 - Resultado do relatório de jogos mais vendidos.

10. Evidências no DB Browser for SQLite

A imagem abaixo mostra a estrutura criada no DB Browser for SQLite. É possível visualizar as tabelas do banco e os dois gatilhos criados para controlar estoque e atualização da venda.



11. Conclusão

O banco de dados desenvolvido atende à proposta da atividade, pois utiliza SQLite, possui tabelas relacionadas, tratamentos de campos, triggers e relatórios SQL funcionais. O tema escolhido permitiu simular um controle básico de uma loja de jogos, envolvendo cadastro de produtos, clientes, vendas, pagamentos e estoque. Durante os testes, as vendas foram registradas e os relatórios retornaram informações úteis para gerenciamento, como vendas por cliente, faturamento por plataforma, separação de jogos físicos e digitais e jogos mais

vendidos. Assim, o projeto demonstra a aplicação prática dos conceitos de banco de dados estudados em sala.